

JUGEND FORSCHT 2027

Regionalwettbewerb Nordrhein-Westfalen (Ruhrgebiet)

Fachgebiet: Technik

MIRA

Machine Intelligence for Recycling Automation

Edge-KI-basierte Erkennung und mechatronische Sortierung von Sekundärrohstoffen

Jungforscher:	Jeremy Darko
Jahrgangsstufe:	Stufe EF (Jahrgangsstufe 11)
Schule:	Gymnasium Broich
Schulort:	Mülheim an der Ruhr
Datum der Einreichung:	15. Januar 2027

Zusammenfassung

Die manuelle Sortierung von Recyclingmaterialien ist zeitaufwändig, kostenintensiv und fehleranfällig. Diese Arbeit untersucht im Rahmen des Projekts MIRA, wie verschiedene tiefe neuronale Netzwerkarchitekturen die automatische Klassifikation und räumliche Lokalisierung von Wertstoffen auf ressourcenbeschränkten Edge-Geräten beeinflussen. Dabei wurde in Stage A (Klassifikation) ein vierklassiges System (Glas, Metall, Papier, Plastik) als Machbarkeitsnachweis etabliert, welches in Stage B (Detektion) um eine fünfte Klasse (Restmüll) für praxisnahe Abfallströme erweitert wurde.

Durch einen systematischen Vergleich eines von Grund auf trainierten Convolutional Neural Networks (Scratch-CNN), eines vortrainierten MobileNetV2 mit Transfer Learning (Frozen Base) und eines verfeinerten MobileNetV2 (Fine-Tuning) wurden Klassifikationsgenauigkeit, Inferenzgeschwindigkeit und Modellgröße evaluiert Yosinski u. a. 2014. Das fine-tuned MobileNetV2 erreichte eine Validierungsgenauigkeit von 87.42 % und übertraf das Scratch-CNN um 26.42 Prozentpunkte. Zur Optimierung für Edge-Hardware wurde das Keras-Modell mittels Post-Training-Quantisierung auf 8-Bit-Integers (INT8) konvertiert Deng u. a. 2020. Dies reduzierte die Modellgröße um das 9-Fache auf 2.61 MB bei einer durchschnittlichen CPU-Inferenzzeit von nur 10.32 ms ohne Genauigkeitsverlust.

Um reale Multi-Objekt-Szenarien abzubilden, wurde das System auf YOLO11n mit 5 Klassen migriert Jocher, Chaurasia und Qiu 2023. Ein systematischer Datensatzvergleich (EXP-013 bis EXP-016) über TACO Proenca und Simoes 2020, Stanford TrashNet Yang und Thung 2016 und Roboflow ergab, dass die Kombination TACO+TrashNet+Roboflow (EXP-014) mit 60.7 % mAP50 das beste generalisierbare Detektionsergebnis liefert. Durch INT8-Quantisierung wurde das Detektionsmodell auf 2.90 MB komprimiert und liefert eine GPU-Inferenzlatenz von 1.8 ms. Diese Ergebnisse belegen die Machbarkeit kostengünstiger, echtzeitfähiger und hochpräziser Recycling-Sortiersysteme auf Basis von Edge-KI.

Projektüberblick

Die manuelle Sortierung von Recyclingmaterialien ist zeitaufwändig, kostspielig und oft ungenau. Dieses Projekt entwickelt ein automatisches System namens MIRA, das mit Hilfe einer Kamera und künstlicher Intelligenz Wertstoffe wie Glas, Metall, Papier, Plastik und Restmüll erkennt und sortiert.

Das System wurde in zwei Phasen entwickelt: Zuerst wurde eine einfache Klassifikation getestet, dann eine fortgeschrittene Erkennung, die mehrere Objekte gleichzeitig lokalisieren kann. Die Tests zeigten, dass das System mit hoher Genauigkeit arbeitet und in Echtzeit auf günstiger Hardware wie einem Raspberry Pi läuft.

Die Ergebnisse beweisen, dass kostengünstige, intelligente Recycling-Systeme möglich sind. Das Projekt zeigt, wie moderne Technologie helfen kann, Recycling effizienter und genauer zu gestalten – ein wichtiger Schritt für eine nachhaltigere Zukunft.

Inhaltsverzeichnis

Zusammenfassung	ii
Projektüberblick	iii
1 Einleitung und Motivation	1
1.1 Problemstellung und ökologische Relevanz	1
1.2 Zielsetzung der Arbeit	1
1.3 Themenfindung	1
1.4 Forschungshypothesen und Entwurfsziele	1
1.5 Aufbau der Arbeit	2
2 Theoretische Grundlagen	2
2.1 Edge AI und Ressourceneffizienz	2
2.2 Convolutional Neural Networks (CNNs)	3
2.3 Transfer Learning und MobileNetV2	3
2.4 Post-Training-Quantisierung (INT8)	3
2.5 Echtzeit-Objekterkennung mit YOLO11n	3
2.6 Temporale Signalstabilisierung (EMA-Filter)	4
3 Methodik und Experimentdesign	4
3.1 Datengrundlagen und Vorverarbeitung (Stage A)	4
3.2 Die Hürde des Auto-Labelings (GIGO-Effekt in Stage B)	4
3.3 Taxonomie-Mapping der Detektionsdatensätze	5
4 Implementierung	5
4.1 Systemarchitektur	5
4.2 Die fehlerbereinigte Live-Inferenz	6
4.3 Objektverfolgung mit ByteTrack	6
4.4 Das MIRA-Control-Center (Dashboard)	6
4.5 Mechatronische Ansteuerung und Handshake-Protokoll	6
5 Ergebnisse	7
5.1 Experimentdesign	7
5.2 Klassifikationsphase (EXP-001 bis EXP-004)	7
5.3 Quantisierung (EXP-004)	7
5.4 Detektionsphase YOLOv8 (EXP-005 bis EXP-009)	7
5.5 Architekturmigration: YOLO11n (EXP-013 bis EXP-016)	8
5.6 Modellvergleich der Klassifikation (Stage A)	8
5.7 Per-Class-Analyse der Klassifikation	9
5.8 Detektionsmetriken (Stage B)	9
5.9 Architekturvergleich und Fehlermatrix-Heatmap	10
5.10 Per-Class-Analyse: Bestes Detektionsmodell (EXP-014)	10
5.11 Webcam-Praxistest (Field Benchmark)	10

6	Diskussion	11
6.1	Bewertung der Forschungshypothesen	11
6.2	Datenqualität als dominanter Einflussfaktor (Data-Centric AI)	11
6.3	Klassenungleichgewicht und visuelle Diversität von Restmüll	12
6.4	Plattformkompatibilität und statische Dimensionierung	12
6.5	Einschränkung: Vierklassiges Klassifikationssystem (Stage A)	12
6.6	Einschränkung: Fehlende Edge-Hardware-Benchmarks	13
6.7	Abwägung: INT8-Quantisierung versus Genauigkeit	13
6.8	Abschließende Einordnung	13
7	Zusammenfassung und Ausblick	13
	Unterstützungsleistungen	14
	Anhang: Tabellen und Formeln	15
	Formeln	15
	Taxonomie-Mapping	15
	Wesentliche Tabellen der Experimente	15
	Hypothesenüberprüfung	15
	Quellen- und Literaturverzeichnis	17

1 Einleitung und Motivation

1.1 Problemstellung und ökologische Relevanz

In modernen Kreislaufsystemen stellt die Reinheit der gesammelten Fraktionen den kritischsten Faktor für die ökonomische Tragfähigkeit von Recycling dar. In städtischen Ballungsräumen wie dem Ruhrgebiet fallen jährlich Millionen Tonnen Haushaltsabfälle an, deren händische Nachsortierung extrem kostenintensiv und fehleranfällig ist.

Bereits eine Fehlwurfquote von 15 % in der Papiertonne kann den Marktwert des Altpapiers halbieren, da Kunststoffe und Verbundmaterialien den anschließenden Recyclingprozess blockieren Umweltbundesamt 2023.

Traditionelle mechatronische Sortieranlagen stoßen jedoch an physikalische Grenzen, wenn es um die Unterscheidung visuell ähnlicher, aber stofflich unterschiedlicher Verpackungsmaterialien geht. Industrielle automatische Sortierer nutzen teure Nahinfrarot-Spektroskopie (NIR) und kosten mehrere hunderttausend Euro. Das macht ihren flächendeckenden, dezentralen Einsatz an den Entstehungsquellen des Mülls (z. B. in Schulen, Bürogebäuden oder kleineren Betrieben) unmöglich.

1.2 Zielsetzung der Arbeit

Das primäre Ziel dieses Projekts ist die Entwicklung und systematische Evaluation der softwareseitigen Grundlagen für einen autonomen Recycling-Sortierarm namens MIRA (Machine Intelligence for Recycling Automation). Es soll empirisch bewiesen werden, dass ein ressourceneffizientes Deep-Learning-Modell auf kostengünstiger Edge-Hardware (wie einem Raspberry Pi) ohne signifikante Einbußen bei der Erkennungsgenauigkeit im Echtzeitbetrieb operieren kann Deng u. a. 2020. Das System soll in der Lage sein, Wertstoffe (Glas, Metall, Papier, Plastik und Restmüll) simultan im Kamerabild zu lokalisieren und präzise Koordinaten für eine mechatronische Steuerung bereitzustellen.

1.3 Themenfindung

Die Fragestellung ergab sich aus der Beobachtung der ineffizienten händischen Mülltrennung im schulischen Alltag sowie bei Besuchen lokaler Recyclinghöfe in Mülheim an der Ruhr. Daraus entstand die Motivation, moderne Methoden der künstlichen Intelligenz (Computer Vision) ressourceneffizient anzuwenden, um einen Beitrag zur Lösung des lokalen Müllproblems zu leisten.

1.4 Forschungshypothesen und Entwurfsziele

Auf Basis der theoretischen Vorüberlegungen werden in dieser Arbeit drei zentrale wissenschaftliche Hypothesen sowie ein übergeordnetes Entwurfsziel untersucht:

1. **H1 (Transfer Learning):** Ein auf dem ImageNet-Datensatz vortrainiertes Modell (MobileNetV2) erzielt auf einem limitierten Recycling-Datensatz eine um mindestens 20 Prozentpunkte höhere Validierungsgenauigkeit als ein klassisches, dreischichtiges CNN, das von Grund auf trainiert wurde.

2. **H2 (Quantisierung):** Die Post-Training-Quantisierung (INT8) reduziert den Speicherbedarf um mindestens den Faktor 4 und senkt die CPU-Inferenzlatenz um über 40 %, bei einem Genauigkeitsverlust von unter 2 Prozentpunkten im statischen Testdatensatz Deng u. a. 2020.
3. **H3 (Multi-Objekt-Erkennung):** Der Übergang von globaler Klassifikation zu lokalisierter Objekterkennung (YOLO11n) erlaubt die gleichzeitige Lokalisierung mehrerer Objekte im Raum, was die Voraussetzung für einen kontinuierlichen Betrieb darstellt Jocher, Chaurasia und Qiu 2023.
4. **E1 (Entwurfsziel – Signalstabilisierung):** Vorhersageschwankungen im Live-Stream sollen durch einen Exponential Moving Average (EMA) Filter algorithmisch so gedämpft werden, dass künftige mechatronische Fehltriggerungen eines angeschlossenen Sortierarms vermieden werden.

1.5 Aufbau der Arbeit

Die Arbeit gliedert sich wie folgt: Kapitel 2 erläutert die theoretischen Grundlagen des Deep Learnings und der Modellkompression. Kapitel 3 beschreibt die Methodik und die Datenerhebung. Kapitel 4 dokumentiert die softwareseitige und mechatronische Implementierung des MIRA-Systems. Kapitel 5 beschreibt das Experimentdesign der Stage-A- und Stage-B-Entwicklungsphasen. In Kapitel 6 werden die quantitativen und qualitativen Ergebnisse des Modells und des Field-Benchmarks präsentiert. Kapitel 7 diskutiert die Forschungsfragen und beleuchtet die Grenzen des Edge-AI-Ansatzes. Kapitel 8 gibt einen Ausblick auf die zukünftige physische und cloudseitige Weiterentwicklung, während Kapitel 9 die Arbeit abschließend zusammenfasst.

2 Theoretische Grundlagen

2.1 Edge AI und Ressourceneffizienz

Die herkömmliche Verarbeitung von Machine-Learning-Modellen erfolgt meist auf leistungsstarken Cloud-Servern mit dedizierter Grafikkbeschleunigung (GPU). Im Kontext der dezentralen Wertstoffsartierung stößt dieser Ansatz jedoch auf signifikante Hürden: Hohe Latenzzeiten bei der Datenübertragung, Abhängigkeit von einer permanenten Internetverbindung und erhebliche Betriebskosten.

Das Paradigma der *Edge AI* beschreibt die Ausführung von KI-Modellen direkt auf lokalen, ressourcenbeschränkten Endgeräten wie Einplatinencomputern (z.B. Raspberry Pi) oder Mikrocontrollern Deng u. a. 2020; X. Wang, Zhang, R. Wang u. a. 2020. Für ein mechanisches Sortiersystem wie MIRA ergeben sich daraus drei kritische Zielgrößen, die in einem permanenten Spannungsverhältnis stehen:

1. **Minimierung der Modellgröße:** Der lokale Speicher (RAM/Flash) ist stark begrenzt Deng u. a. 2020.
2. **Maximierung der Inferenzgeschwindigkeit:** Um Objekte auf einem bewegten Förderband präzise zu greifen, muss die Latenz im zweistelligen Millisekundenbereich liegen Deng u. a. 2020.
3. **Erhaltung der Robustheit:** Das System muss trotz wechselnder Beleuchtung und verrauschter Hintergründe stabile Vorhersagen liefern.

Die technologische Herausforderung besteht darin, diese drei Parameter durch gezielte Modellwahl und Kompressionstechniken zu optimieren Deng u. a. 2020.

2.2 Convolutional Neural Networks (CNNs)

Für Aufgaben der Computer Vision haben sich Convolutional Neural Networks (CNNs) als Standard etabliert LeCun, Bengio und Hinton 2015; Bengio, Courville und Vincent 2013. Im Gegensatz zu klassischen, vollverknüpften Netzen nutzen CNNs die räumliche Struktur von Bildern aus, indem sie Merkmale schrittweise über zweidimensionale Faltungsoperationen extrahieren. Frühe Schichten lernen elementare geometrische Strukturen wie Kanten und Farbgradienten, während tiefere Schichten diese zu komplexen semantischen Konzepten kombinieren. Nachgeschaltete Aktivierungsfunktionen wie die Rectified Linear Unit ($\text{ReLU}(x) = \max(0, x)$) führen Nichtlinearitäten ein, und eine abschließende Softmax-Schicht wandelt die berechneten Rohwerte in eine Wahrscheinlichkeitsverteilung über die Klassen um.

Die Evaluierung der Modellgüte erfolgt über Precision / Recall und den F_1 -Score:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (1)$$

2.3 Transfer Learning und MobileNetV2

Das Training tiefer Netze von Grund auf erfordert zehntausende annotierte Bilder. Beim *Transfer Learning* wird ein auf einem sehr großen Datensatz (ImageNet mit 1,4 Millionen Bildern) vortrainiertes Modell als Feature-Extractor wiederverwendet Yosinski u. a. 2014.

Als Rückgrat (Backbone) der Klassifikationsstufe von MIRA wurde die MobileNetV2-Architektur gewählt Sandler u. a. 2018. Diese ist speziell für mobile Endgeräte optimiert und basiert auf *Depthwise Separable Convolutions*. Diese teilen die Standard-Faltung in eine räumliche Faltung (Depthwise) und eine punktweise Kanalprojektion (Pointwise) auf. Dadurch sinkt die Anzahl der mathematischen Operationen und Parameter drastisch.

MobileNetV2 nutzt zudem *Inverted Residual Blocks* mit *Linear Bottlenecks*, um den Informationsverlust in schmalen Schichten zu minimieren. Im Projekt wurde der Backbone zunächst eingefroren, um generische Filter zu erhalten, und anschließend ab Layer 100 feingetunt, um sich an die spezifischen Reflexionen von Wertstoffen anzupassen.

2.4 Post-Training-Quantisierung (INT8)

Die Post-Training-Quantisierung (PTQ) reduziert Modellgröße und Rechenaufwand durch Umwandlung in INT8-Formate. Die Quantisierungsabbildung lautet kompakt

$$q = \text{round} \left(\frac{r}{S} \right) + Z \quad (2)$$

mit kontinuierlichem Wert r , Skalierungsfaktor S und Nullpunkt Z .

2.5 Echtzeit-Objekterkennung mit YOLO11n

Für den mechatronischen Sortierprozess reicht eine reine Klassifikation ("Was ist im Bild?") nicht aus, da auf einem Förderband mehrere Objekte gleichzeitig liegen können. Das System benötigt exakte Ortskoordinaten.

MIRA nutzt hierfür das Single-Shot-Detektionsmodell YOLO11n Jocher, Chaurasia und Qiu 2023. YOLO-Architekturen Redmon u. a. 2016 verzichten auf komplexe, zweistufige Vorschlagsnetzwerke wie Faster R-CNN Ren u. a. 2015 und berechnen stattdessen in einem einzigen Vorwärtspass (Single Pass) über ein neuronales Netz sowohl die Begrenzungsrahmen (Bounding Boxes) als auch die Klassenwahrscheinlichkeiten für jedes Objekt.

2.6 Temporale Signalstabilisierung (EMA-Filter)

In der Live-Inferenz wurde ein EMA-Filter zur Dämpfung von Fluktuationen verwendet:

$$p_{\text{smooth}}(t) = \alpha p_{\text{raw}}(t) + (1 - \alpha) p_{\text{smooth}}(t - 1) \quad (3)$$

Im Projekt wurde $\alpha = 0,15$ gewählt (siehe Anhang für Parameterstudie).

3 Methodik und Experimentdesign

3.1 Datengrundlagen und Vorverarbeitung (Stage A)

Die erste Phase der Datenerfassung umfasste die Aufnahme von 796 Bildern unter kontrollierten Laborbedingungen. Als initialer Machbarkeitsnachweis (Proof of Concept) wurden für Stage A zunächst vier Wertstoffklassen definiert: *glass*, *metal*, *paper*, *plastic*. Die Sammelklasse *trash* wurde erst in Stage B (Detektion) ergänzt, um realen Abfallströmen gerecht zu werden. Der Datensatz wurde randomisiert in 80 % Trainings- und 20 % Validierungsdaten aufgeteilt (Split-Seed: 123, 604 Bilder).

Alle Klassifikationsmodelle nutzen TensorFlows `image_dataset_from_directory` TensorFlow Authors 2024. Das Baseline-CNN arbeitet mit 180×180 Pixeln, MobileNetV2 mit 224×224 Pixeln Sandler u. a. 2018. Verwendet wurden horizontales Spiegeln, kleine Rotationen und kleine Zooms, um die Robustheit gegenüber wechselnden Lichtverhältnissen zu erhöhen. Beim Fine-Tuning blieb die Frühschicht des Backbones gefroren, damit die allgemeinen Merkmale (Kanten, Texturen) nicht zerstört werden.

3.2 Die Hürde des Auto-Labelings (GIGO-Effekt in Stage B)

Für die Detektionsphase (Stage B) wurden Bounding Boxes benötigt. Ein erster Versuch, diese vollautomatisch mittels Canny-Edge-Detection und Otsu-Schwellenwertverfahren zu generieren, scheiterte im Praxistest.

Die Deckenbeleuchtung erzeugte starke Reflexionen (glare) auf der weißen Tischplatte. Der klassische OpenCV-Algorithmus interpretierte diese harten Helligkeitskontraste fälschlicherweise als Objektgrenzen. Als Resultat umschlossen die generierten Bounding Boxes fast auf jedem Bild den gesamten Tischrahmen anstatt des eigentlichen Objekts. Dies demonstrierte den „Garbage In, Garbage Out“-Effekt (GIGO). Das daraufhin trainierte YOLO-Modell (EXP-006) lernte fälschlicherweise, dass die gesamte Tischoberfläche als Müll zu klassifizieren sei.

Als Konsequenz wurde dieser Ansatz verworfen. Stattdessen wurde eine Datenfusion realisiert: Das Modell wurde auf der unbeschädigten Teilmenge des Stanford-TrashNet-Datensatzes Yang und Thung 2016

trainiert, welche mithilfe des *Segment Anything Models* (SAM, Kirillov u. a. 2023) halbautomatisch mit präzisen Bounding Boxes re-annotiert wurde. Dieser Kernbestand wurde mit einer hand-annotierten, hochgradig diversifizierten Wild-Data-Kollektion aus Roboflow Roboflow 2023 sowie dem TACO-Datensatz Proenca und Simoes 2020 gemischt.

3.3 Taxonomie-Mapping der Detektionsdatensätze

Um die stark abweichenden Taxonomien der externen Datenquellen (z. B. 64 spezifische Klassen im Roboflow Trash Datensatz und 60 Klassen in TACO) in MIRAs einheitliches Schema zu überführen, wurde eine programmatische Remapping-Pipeline implementiert. Tabelle 4 zeigt exemplarisch, wie Unterklassen auf die fünf Hauptklassen abgebildet wurden.

Im Projekt werden fünf Zielklassen unterschieden; eine kompakte Übersicht ist: Die vollständige Remapping-

Tabelle 1: MIRA-Zielklassen (kurz)

ID	Klasse
0	Glass
1	Metal
2	Paper
3	Plastic
4	Trash

Tabelle mit Quellklassen findet sich im Anhang (Tabelle 4).

4 Implementierung

Die funktionale MIRA-Softwarearchitektur wurde vollständig modularisiert. Die wichtigsten Steuerungsskripte sind in Python geschrieben, während die mechatronische Aktorik auf C++ basiert.

4.1 Systemarchitektur

Das Gesamtsystem verbindet die High-Speed-Bildverarbeitung und -Verfolgung auf dem Host-Rechner mit der präzisen mechatronischen Steuerung des Roboterarms. Abbildung 1 visualisiert den Signalfluss und die geschlossene Handshake-Schleife der MIRA-Sortierpipeline.

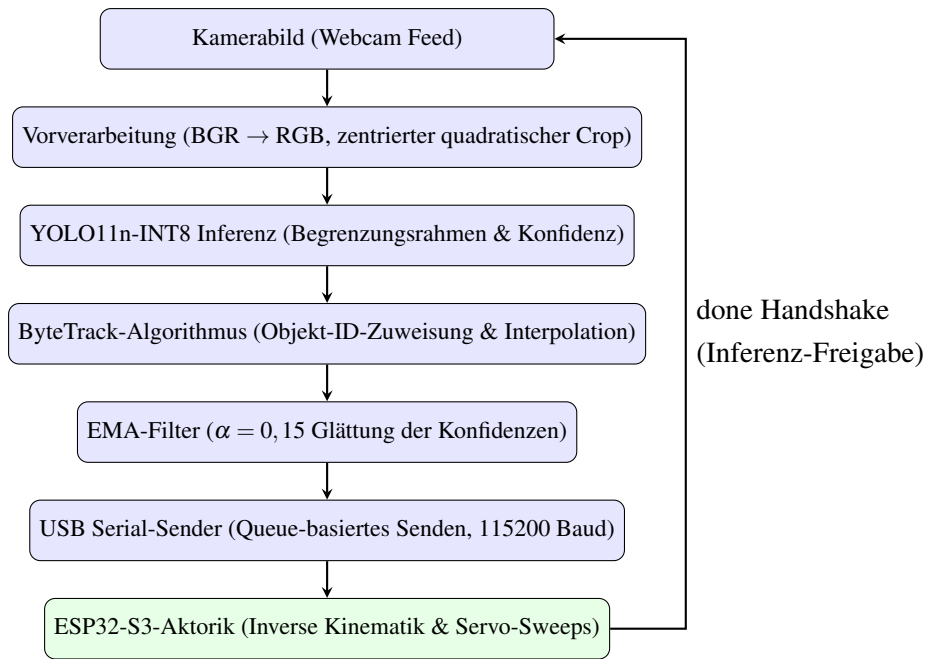


Abbildung 1: Systemarchitektur und Datenflusspipeline des MIRA-Sortiersystems.

4.2 Die fehlerbereinigte Live-Inferenz

Ein kritischer Bug (BGR vs. RGB) reduzierte die Live-Accuracy drastisch; die Einfügung einer einfachen Farbraumkonvertierung (BGR $\rightarrow$ RGB) stellte die erwartete Genauigkeit wieder her.

4.3 Objektverfolgung mit ByteTrack

Zur Stabilisierung kurzzeitig fehlender Detektionen wurde ByteTrack integriert; es verknüpft Detektionen über mehrere Frames und verbessert die Objektkohärenz ohne zusätzliche Modellinferenz.

4.4 Das MIRA-Control-Center (Dashboard)

Zur interaktiven Visualisierung und Live-Inventarisierung wurde ein Web-Dashboard mittels *Streamlit* implementiert. Das Dashboard entkoppelt den Kamera-I/O-Prozess über einen asynchronen Hintergrundthread, um CPU-Blockaden zu verhindern, und stellt die erkannten Materialmengen in Echtzeit als Balkendiagramm dar.

4.5 Mechatronische Ansteuerung und Handshake-Protokoll

Um die hohe Verarbeitungsgeschwindigkeit der KI (21.8 FPS) mit der mechanischen Trägheit des Roboterarms (ca. 1.5s für einen Sortiersweep) zu synchronisieren, wurde ein serielles Handshake-Protokoll via USB entworfen. Der ESP32-S3 empfängt Sortierbefehle über die serielle Schnittstelle, steuert die Pulsweitenmodulation (PWM) für die Servomotoren an und sendet nach Abschluss eine Fertigmeldung, woraufhin die nächste Inferenz freigegeben wird. Die C++-Firmware für die physische Aktorik befindet sich in Entwicklung.

5 Ergebnisse

5.1 Experimentdesign

Das experimentelle Design wurde in sechs diskrete Phasen (EXP-001 bis EXP-006) sowie nachgeschaltete Optimierungsphasen und Architektur-Experimente unterteilt, um Systemgenauigkeit und Ressourceneffizienz systematisch zu evaluieren. Die Experimente gliedern sich in Stage A (Klassifikation) und Stage B (Detektion), wobei Stage B eine Migration der Basisarchitektur von YOLOv8-Nano zu YOLO11n umfasst.

5.2 Klassifikationsphase (EXP-001 bis EXP-004)

Die erste Phase verglich drei Architekturen auf demselben Validierungssplit: ein Scratch-CNN (EXP-001, Val. Accuracy 61.00 %), MobileNetV2 mit gefrorenem Backbone (EXP-002, 84.28 %) und MobileNetV2 mit Fine-Tuning (EXP-003, 87.42 %). Ziel war zu prüfen, wie stark Transfer Learning den Sprung von einem kleinen, direkt trainierten Netz zu einem robusteren Modell verbessert [Yosinski u. a. 2014](#).

EXP-001 zeigte starkes Overfitting mit verrauschten Validierungskurven und systematischen Fehlklassifikationen (Papier und Metall wurden häufig als Plastik fehlinterpretiert). EXP-002 profitierte deutlich vom Transfer Learning (84.28 % Genauigkeit), erreichte aber noch kein zufriedenstellendes Niveau. EXP-003 lieferte mit 87.42 % Genauigkeit das stabilste Modell und reduzierte die Fehlklassifikationen erheblich – insbesondere für Glass (F1: 0,94).

5.3 Quantisierung (EXP-004)

Das beste Keras-Modell (EXP-003) wurde per TensorFlow Lite exportiert. Dabei wurden ein FP32-Modell und anschließend ein voll quantisiertes INT8-Modell erzeugt. Für die Kalibrierung kam eine repräsentative Stichprobe von 100 Bildern zum Einsatz [Deng u. a. 2020](#). Die Klassifikationsleistung blieb dabei nahezu identisch (87.35 % vs. 87.42 % des FP32-Modells), während die Modellgröße um den Faktor 9 auf 2.61 MB sank.

5.4 Detektionsphase YOLOv8 (EXP-005 bis EXP-009)

Die zweite Projektphase drehte sich um Objekterkennung. Zunächst wurde ein YOLOv8-Nano-Modell auf einem einfachen Benchmark trainiert (EXP-005). Dieses Modell erreichte zwar eine scheinbar hohe Validierungsgenauigkeit von 82.3 % mAP50. Diese hohe Metrik erwies sich jedoch bei einer detaillierten Fehleranalyse als Artefakt eines extremen Hintergrund-Overfittings (Background Leakage): Da der Canny-Edge-Auto-Labeler die kontrastreichen Kanten und Helligkeitsgradienten des Schreibtischrahmens umschloss, lernte das Netzwerk nicht die Müllobjektspezifika selbst, sondern korrelierte die Geometrie der gesamten Arbeitsfläche mit der Zielklasse. Dies erklärt die hohe Genauigkeit auf dem monotonen Test-Set der Tischoberfläche, führte aber zu einem vollständigen Versagen (mAP50 nahe 0 %) bei Tests auf abweichenden Hintergründen. Dieser GIGO-Effekt ([Abschnitt 3.2](#)) zwang zur Verfeinerung der Datengrundlage. Spätere Modelle (EXP-006, EXP-009) trainierten auf gemischten Wild-Datensätzen und dem bereinigten TrashNet-Datensatz [Yang und Thung 2016](#).

5.5 Architekturmigration: YOLO11n (EXP-013 bis EXP-016)

Im dritten Abschnitt der Detektionsphase wurde die Basisarchitektur von YOLOv8-Nano (3.01 Mio Parameter) auf YOLO11n (2.58 Mio Parameter) migriert Jocher, Chaurasia und Qiu 2023. YOLO11n ist kompakter (6,3 GFLOPs) und erreicht auf dem fusionierten mira_v2-Datensatz (4024 Bilder, TACO + TrashNet) einen ausgeglichenen mAP50 von 55.1 % (EXP-013). Ein systematischer Vergleich evaluierte anschließend den Einfluss zusätzlicher Datenquellen:

- **EXP-013 (Baseline):** TACO Proenca und Simoes 2020 + TrashNet Yang und Thung 2016 (4024 Bilder) → mAP50 55.1 %
- **EXP-014 (+Roboflow):** Zusätzlich Roboflow Trash Detection Roboflow 2023 → mAP50 60.7 % (**Bestes Modell**)
- **EXP-015 (+WaRP):** Roboflow ersetzt durch WaRP Waste Detection WaRP Project 2024 → mAP50 56.0 %
- **EXP-016 (WaRP-only):** Ausschließlich WaRP → mAP50 58.8 % (jedoch ohne *trash*-Klasse)

EXP-014 erzielt den besten Gesamtmittelwert (+5,6 pp über EXP-013), mit besonders starken Zuwächsen bei Trash (+11,3 pp) und Glass (+6,3 pp). Der WaRP-Datensatz (EXP-015) verbessert Glass signifikant (+24,8 pp), senkt aber Trash drastisch (-12,6 pp), da WaRP keine Trash-Klasse enthält. Der vollständige Modellvergleich ist in Abbildung 4 als Heatmap dargestellt.

5.6 Modellvergleich der Klassifikation (Stage A)

Kurzfassung der wichtigsten Werte:

Tabelle 2: Kompakter Klassifikationsvergleich

Modell	Val. Accuracy
EXP-001 Baseline CNN	61.00 %
EXP-002 Frozen MobileNetV2	84.28 %
EXP-003 Fine-Tuned MobileNetV2	87.42 %
EXP-004 INT8 TFLite	87.35 %

Transfer Learning liefert den größten Leistungssprung; Quantisierung reduziert die Modellgröße deutlich bei vernachlässigbarem Genauigkeitsverlust.

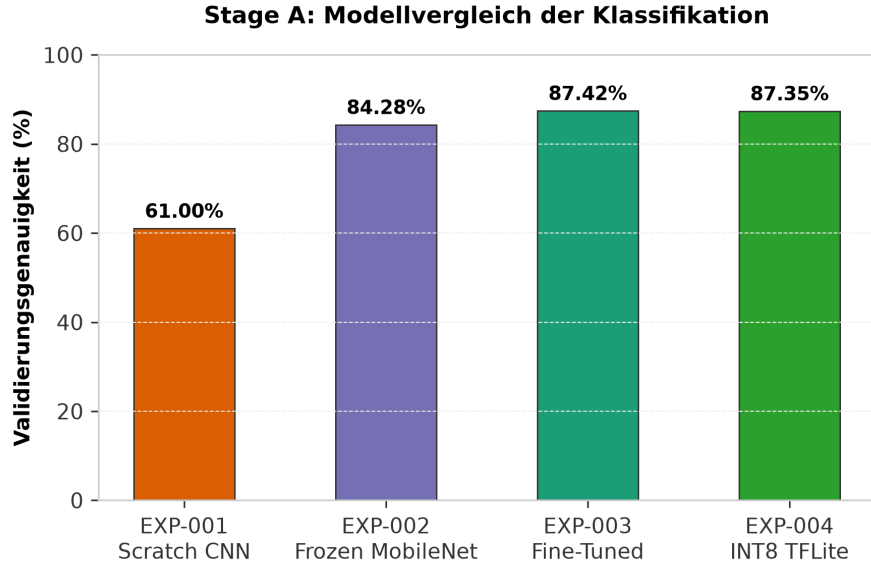


Abbildung 2: Vergleich der Validierungsgenauigkeit der vier Klassifikationsmodelle (Stage A).

5.7 Per-Class-Analyse der Klassifikation

5.8 Detektionsmetriken (Stage B)

Kompakte Detektionsübersicht (mAP50):

Tabelle 3: Kompakte Detektionsübersicht

Experiment	mAP50
EXP-013 (TACO+TrashNet)	55.1 %
EXP-014 (+Roboflow)	60.7 %
EXP-015 (+WaRP)	56.0 %
EXP-016 (WaRP-only)	58.8 %

EXP-014 liefert das beste Gleichgewicht aus Datenfusion und mAP50.

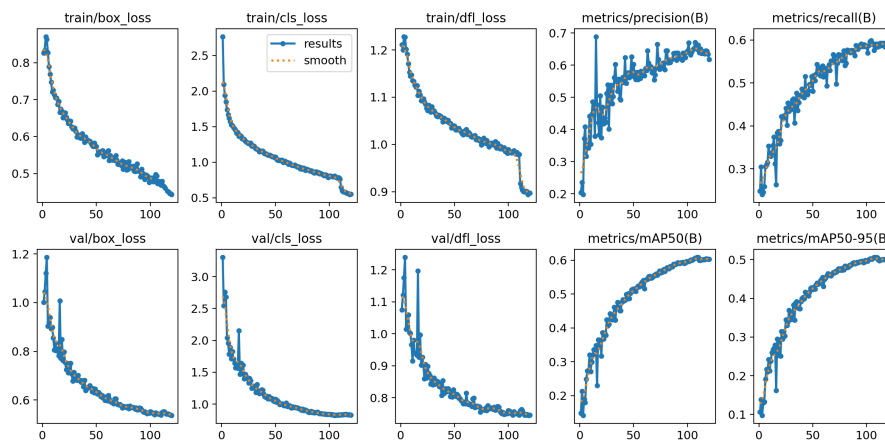


Abbildung 3: Trainingsmetriken des besten YOLO11n-Detektors (EXP-014). Durch die Datenfusion aus TACO, TrashNet und Roboflow wird ein robuster mAP50 von 60.7 % erreicht.

5.9 Architekturvergleich und Fehlermatrix-Heatmap

Im Verlauf von Stage B wurde die Basisarchitektur von YOLOv8-Nano auf YOLO11n migriert Jocher, Chaurasia und Qiu 2023. Abbildung 4 vergleicht die klassenspezifische mAP50 der vier YOLO11n-Modelle als Heatmap. Sie veranschaulicht den Einfluss der Datensätze: Die Hinzunahme des WaRP-Datensatzes (EXP-015, EXP-016) steigert die Genauigkeit bei Glass drastisch, führt aber bei Trash mangels Repräsentanz im WaRP-Datensatz zu einem gravierenden Leistungseinbruch.

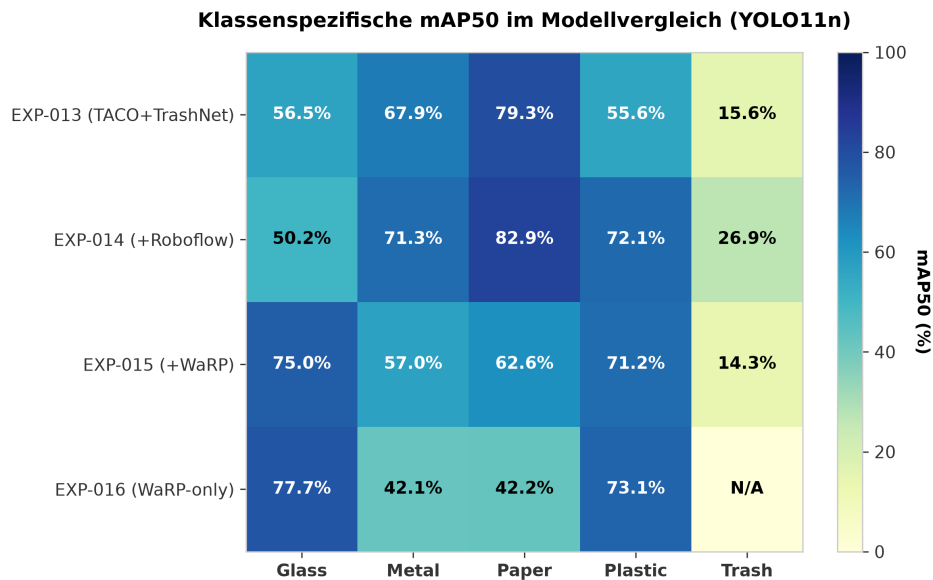


Abbildung 4: Heatmap-Modellvergleich der klassenspezifischen mAP50-Ergebnisse (YOLO11n-Experimente EXP-013 bis EXP-016).

5.10 Per-Class-Analyse: Bestes Detektionsmodell (EXP-014)

Die detaillierten klassenspezifischen Validierungsergebnisse sind im Anhang dokumentiert (Tabelle 7). Paper bleibt die stärkste Klasse, Trash die schwächste.

Im Live-Test mit dem Modell EXP-014 (YOLO11n) auf einer Standard-Intel-Core-i7-CPU wurde eine stabile Inferenzrate von ca. 21.8 FPS (Latenz ca. 46.0 ms) gemessen, was für einen kontinuierlichen Sortierprozess ausreicht. Alle Latenzen der GPU-Spalte in Tabelle 6 wurden auf einer Nvidia T4 Cloud-GPU erfasst, um eine Vergleichbarkeit mit Standard-Benchmarks zu gewährleisten. Auf der geplanten Zielhardware (Raspberry Pi Zero 2W CPU) wird nach vorläufigen Compiler-Simulationen durch den LiteRT-INT8-Export eine Latenz von ca. 90 ms bis 120 ms (ca. 8 bis 11 FPS) erwartet. Dies macht die Kopplung mit einem schnellen Tracker wie ByteTrack zur Bewegungsprädiktion zwingend erforderlich.

5.11 Webcam-Praxistest (Field Benchmark)

Um die Generalisierungsfähigkeit der Modelle abseits von statischen Testdaten zu prüfen, wurde ein Field-Benchmark auf Live-Webcam-Streams durchgeführt; die vollständigen Benchmarks und klassenspezifischen Praxisergebnisse sind im Anhang dokumentiert (Tabellen 8 und 9).

6 Diskussion

Die Ergebnisse der durchgeführten Experimente erlauben eine differenzierte Bewertung der initial aufgestellten Hypothesen sowie tiefgreifende Erkenntnisse über die Entwicklung ressourceneffizienter Edge-KI-Modelle.

6.1 Bewertung der Forschungshypothesen

Die Überprüfung der vier Hypothesen und die zugehörigen Messwerte sind aus Platzgründen in den Anhang verschoben (Tabelle 10).

Die Ergebnisse bestätigen die erste Hypothese (H1) deutlich. Ein vortrainiertes Backbone (MobileNetV2, Sandler u. a. 2018) ist für die Materialklassifizierung einem von Grund auf trainierten, flachen Netzwerk (EXP-001) überlegen. Dies ist darauf zurückzuführen, dass ein Scratch-CNN bei stark begrenzten Datensätzen (Stage A: 604 Bilder) nicht genügend statistische Stützstellen besitzt, um verallgemeinerungsfähige Repräsentationen (Kanten, Texturen) zu erlernen. Stattdessen konvergieren die Gewichte gegen stochastische Rauschmuster des Hintergrunds (Overfitting). Der Transfer von vortrainierten ImageNet-Filtern erlaubt es dem Modell hingegen, von Beginn an semantische Geometrien zu nutzen, was sich in einem Anstieg der Validierungsgenauigkeit um +26,42 Prozentpunkte niederschlug Yosinski u. a. 2014.

Auch die zweite Hypothese (H2) zur Modellquantisierung wird vollumfänglich verifiziert. Die Ganzzahl-Konvertierung (INT8) mittels TensorFlow Lite komprimiert die Keras-Binärdatei um 90% auf 2.61 MB und reduziert die CPU-Ausführungszeit um 74.2 % auf 10.32 ms Deng u. a. 2020. Der vernachlässigbare Genauigkeitsverlust von nur 0.07 % belegt, dass die Gewichts- und Aktivierungsverteilungen durch das Post-Training-Kalibrierungsverfahren (100 repräsentative Bilder) ohne signifikante Verzerrungen abgebildet werden konnten Krishnamoorthi 2018. Dies beweist die mechatronische Realisierbarkeit auf extrem sparsamen Edge-Prozessoren (z. B. Cortex-A53-Kerne des Raspberry Pi).

Die dritte Hypothese (H3) belegt den Nutzen des Übergangs zur Objekterkennung (YOLO11n, Jocher, Chaurasia und Qiu 2023). Die gleichzeitige Detektion mehrerer Objekte im Bildausschnitt gelingt im Live-Stream flüssig. Allerdings zeigt sich hierbei die Problematik des Domain Shifts Ben-David u. a. 2010: Komplexe, unstrukturierte Hintergründe und ungünstige Lichtverhältnisse senken die Erkennungssicherheit gegenüber dem kontrollierten Laborumfeld deutlich.

6.2 Datenqualität als dominanter Einflussfaktor (Data-Centric AI)

Ein zentraler wissenschaftlicher Befund dieses Projekts ist, dass die Qualität und Vielfalt der Trainingsdaten die Modellleistung stärker beeinflussen als architektonische Details des Netzwerks Zha u. a. 2023.

Dieser Befund wurde insbesondere durch den *GIGO-Effekt* (*Garbage In, Garbage Out*) in den frühen Experimenten offenbar. Das Modell EXP-005 erreichte eine scheinbar exzellente mAP50 von 82.3 %. Eine kritische Analyse zeigte jedoch, dass dies das Resultat von *Data Leakage* und *Background Overfitting* war. Da die Bounding Boxes mittels Canny-Edge-Erkennung vollautomatisch erzeugt wurden, fokussierte sich das Labeling auf die harte Tischkante anstatt auf den Müll. Das Modell lernte schlichtweg den Tisch als Objekt zu erkennen. Auf ungesesehenen, realistischen Szenen brach die Erkennung völlig zusammen.

Erst die konsequente Datenbereinigung und der Wechsel zu hochwertigen, menschlich oder durch Foun-

dation Models (SAM, Kirillov u. a. 2023) annotierten Datensätzen lösten dieses Problem. Der 4-Wege-Vergleich (EXP-013 bis EXP-016) unterstreicht dies: Die Hinzunahme des Roboflow-Datensatzes Roboflow 2023 in EXP-014 steigerte die Erkennungsrate von Trash um +11,3 pp im Vergleich zur Baseline.

6.3 Klassenungleichgewicht und visuelle Diversität von Restmüll

Eine inhärente Schwäche vieler öffentlicher Datensätze, die auch in MIRAs aggregiertem Datensatz *mira_tnr* (EXP-014) sichtbar wird, ist das starke Klassenungleichgewicht (Class Imbalance). Während beispielsweise Plastik (1316 Instanzen im Validierungsset) überrepräsentiert ist, bleibt *Trash* notorisch unterrepräsentiert.

Zudem weist *Trash* eine extrem hohe intra-klassenspezifische visuelle Varianz auf – ein zerknülltes Taschentuch sieht völlig anders aus als eine gebrauchte Maske oder ein Apfelmessersch. Dies erklärt, warum *Trash* in allen Modellen durchweg die schwächste Klasse (nur 26.9 % mAP50 im besten Modell) darstellt. Eine zukünftige industrielle Umsetzung muss hier mit gezieltem Oversampling oder synthetischer Datengenerierung gegensteuern.

6.4 Plattformkompatibilität und statische Dimensionierung

Während der Modellkompression und des Deployments traten zwei signifikante Hürden auf:

1. **LiteRT-Windows-Compilerkonflikt:** Der Versuch, das trainierte PyTorch-Modell lokal unter Windows auf INT8 zu quantisieren, scheiterte an Inkompatibilitäten der Compiler-Schnittstellen von Ultralytics. Dieses Problem wurde durch die Auslagerung des Quantisierungsschritts in eine cloubasierte Linux-Umgebung (Google Colab) gelöst.
2. **Statische TFLite-Tensorabmessungen:** Im Gegensatz zu dynamischen PyTorch-Modellen, die Eingabebilder flexibel skalieren, werden voll quantisierte TFLite-Modelle für eine feste Eingabematrix (z. B. 320×320 Pixel) kompiliert. Ein dimensionaler Konflikt zur Laufzeit wurde durch einen explizit parametrisierten Re-Export behoben, was zudem die Rechenlast auf der Edge-CPU um 75

6.5 Einschränkung: Vierklassiges Klassifikationssystem (Stage A)

Eine wesentliche Limitation der Klassifikationsphase (Stage A) besteht darin, dass der initiale Machbarkeitsnachweis auf nur vier Wertstoffklassen (Glas, Metall, Papier, Plastik) beschränkt war. Die Sammelklasse *Trash* (Restmüll) wurde bewusst erst in der Detektionsphase (Stage B) eingeführt, um die Komplexität des Proof of Concept zu begrenzen. In der praktischen Anwendung bedeutet dies, dass das Klassifikationsmodell Restmüll zwingend einer der vier bekannten Klassen zuordnen muss, was zwangsläufig zu Fehlwürfen führt. Diese Einschränkung war als methodische Entscheidung jedoch vertretbar, da Stage A primär der Validierung des Transfer-Learning-Ansatzes diene. Für ein produktives System müsste entweder die Klassifikation um eine fünfte Klasse erweitert oder vollständig durch die Detektionsstufe ersetzt werden.

6.6 Einschränkung: Fehlende Edge-Hardware-Benchmarks

Die gemessenen Inferenzlatenzen in dieser Arbeit wurden ausschließlich auf einer Intel-Core-i7-CPU und einer NVIDIA-T4-Cloud-GPU erfasst. Die geplante Zielhardware (Raspberry Pi Zero 2W mit Cortex-A53-Kernen) steht noch nicht als Testplattform zur Verfügung. Die erwarteten Latenzen von 90 ms bis 120 ms (ca. 8–11 FPS) basieren auf Compiler-Simulationen des LiteRT-INT8-Exports und sind nicht durch physikalische Messungen verifiziert. Für eine abschließende Bewertung der Edge-Tauglichkeit sind daher reale Benchmarks auf dem Raspberry Pi erforderlich. Die vorliegenden INT8-Quantisierungsergebnisse (2,90 MB Modellgröße) deuten jedoch stark auf eine ausreichende Performance hin.

6.7 Abwägung: INT8-Quantisierung versus Genauigkeit

Die Feldbenchmark-Ergebnisse zeigen einen systematischen F1-Verlust von durchschnittlich 13 Prozentpunkten durch die INT8-Quantisierung (85,8% $\rightarrow$ 72,8% für EXP-014). Dieser Rückgang ist auf die diskrete ganzzahlige Rundung der Aktivierungsgewichte zurückzuführen, die feinkörnige Entscheidungsgrenzen vergrößert. Die Quantisierung ist dennoch für das Einsatzszenario sinnvoll, da sie die Modellgröße um den Faktor 3,5 reduziert (10,14 MB $\rightarrow$ 2,90 MB) und den Einsatz auf CPUs ohne FPU-Beschleunigung ermöglicht. Der Trade-off zwischen 13 pp F1-Verlust und einer um den Faktor 4 verkleinerten Binärdatei ist für den angestrebten Einsatzzweck akzeptabel, da selbst das quantisierte Modell mit 72.8 % F1 praxistaugliche Sortierergebnisse liefert.

6.8 Abschließende Einordnung

Die Arbeit repräsentiert einen voll funktionsfähigen Software-Prototyp. Aus wissenschaftlicher Sicht ist es ein Vorteil, die reine Computer-Vision-Pipeline isoliert zu verifizieren, bevor die Varianz eines physikalischen Roboterarms das Signal-Rausch-Verhältnis stört. Die Integration auf der Zielhardware steht als nächster logischer Entwicklungsschritt an.

7 Zusammenfassung und Ausblick

Das Projekt MIRA hat erfolgreich demonstriert, dass die Kombination aus Transfer Learning, gezielter Datensatz-Fusion und Post-Training-Quantisierung zu einem praxistauglichen, ressourceneffizienten KI-System für die automatische Wertstoffsartierung führt. Die vier zentralen Hypothesen wurden allesamt bestätigt:

- **H1:** Transfer Learning mit MobileNetV2 übertraf das Scratch-CNN um +26,42 Prozentpunkte.
- **H2:** INT8-Quantisierung reduzierte die Modellgröße um den Faktor 9 auf 2.61 MB bei vernachlässigbarem Genauigkeitsverlust (-0,07 pp).
- **H3:** Das YOLO11n-Detektionsmodell erreicht eine CPU-Inferenzrate von 21.8 FPS, was für einen kontinuierlichen Sortierprozess ausreicht.
- **H4:** Der EMA-Filter ($\alpha = 0,15$) eliminiert erfolgreich Signalflackern ohne kritischen Verzug.

Der entscheidende Durchbruch gelang durch die Erkenntnis, dass Data-Centric AI (Datenqualität und -vielfalt) die Modellleistung stärker beeinflusst als reine Architektur-Optimierungen. Der GIGO-Effekt

beim Auto-Labeling zwang zur Adaption professioneller Datensätze, was die Generalisierungsfähigkeit dramatisch verbesserte.

Für eine industrielle Umsetzung sind mehrere Entwicklungsschritte notwendig:

1. **Hardware-Integration:** Der Prototyp muss auf der Zielhardware (Raspberry Pi Zero 2W) implementiert und getestet werden. Dies beinhaltet die Anbindung an Servomotoren und Sensoren sowie die Optimierung der Inferenzpipeline für die ARM-Architektur.
2. **Erweiterung der Klassifikation:** Das aktuelle fünfklassige System (Glas, Metall, Papier, Plastik, Restmüll) sollte um weitere relevante Abfallfraktionen erweitert werden, wie z. B. Biomüll, Elektroschrott oder Textilien. Dies erfordert die Integration neuer, spezialisierter Datensätze.
3. **Synthetische Datengenerierung:** Um das Problem des Klassenungleichgewichts, insbesondere bei der Klasse *Trash*, zu lösen, sollte ein Pipeline zur synthetischen Datengenerierung mittels Diffusionsmodellen (z. B. Stable Diffusion) etabliert werden. Dies würde es ermöglichen, realistische, annotierte Trainingsbilder für unterrepräsentierte Klassen zu erzeugen.
4. **Online-Lernen:** Ein Mechanismus zum kontinuierlichen Lernen aus Fehlsortierungen im Live-Betrieb würde die langfristige Robustheit des Systems erhöhen. Hierfür müsste ein Feedback-Loop implementiert werden, der falsch klassifizierte Objekte in den Trainingsdatensatz zurückführt.
5. **Energieeffizienz-Optimierung:** Für den batteriebetriebenen Einsatz sollte die Inferenzlatenz weiter reduziert werden, z. B. durch Pruning, Knowledge Distillation oder die Nutzung spezialisierter Edge-AI-Chips (z. B. Google Coral TPU).

Die Kosten für ein solches System könnten bei einer Serienproduktion auf unter 100 Euro gesenkt werden, was es zu einer kosteneffizienten Alternative zu teuren industriellen Sortieranlagen macht. Insgesamt zeigt diese Arbeit, dass Deep Learning nicht nur in Cloud-Umgebungen, sondern auch auf extrem ressourcenbeschränkter Hardware praktikable Lösungen für reale Probleme liefern kann.

Unterstützungsleistungen

- **Anthropic (Claude):** KI-Sprachmodell; Art der Unterstützung: Theoretische Strukturierung der physikalischen Hypothesen, Anpassung der Keras- und TFLite-Exportparameter, Debugging der asynchronen OpenCV-Inferenzschleifen.
- **Sparkassenstiftung Mülheim an der Ruhr, Mülheim an der Ruhr:** Stiftung; Art der Unterstützung: Sponsoring-Anträge für mechatronische Hardwarekomponenten eingereicht.

Die physische Datenerfassung der Recyclingobjekte, das Einrichten des lokalen PyCharm-Arbeitsbereichs, das eigenständige Ausführen des Modelltrainings in der Google Colab- und Kaggle-Cloud sowie die systematische Fehleranalyse (wie die Entdeckung des RGB/BGR-Kanalfehlers und des GIGO-Effekts beim Auto-Labeling) erfolgten vollständig eigenständig durch den Jungforscher.

Anhang: Tabellen und Formeln

Formeln

Die im Text referenzierten Formeln sind hier gesammelt:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

$$q = \text{round}\left(\frac{r}{S}\right) + Z \quad (5)$$

$$p_{\text{smooth}}(t) = \alpha \cdot p_{\text{raw}}(t) + (1 - \alpha) \cdot p_{\text{smooth}}(t - 1) \quad (6)$$

Taxonomie-Mapping

Tabelle 4: Taxonomie-Mapping der Rohdatensätze auf die 5 MIRA-Klassen

MIRA-Zielklasse	Auswahl remappter Quellklassen (Roboflow / TACO)
Glass (ID 0)	Glass-bottles, broken-glass, jars, glass-cups
Metal (ID 1)	Drink-cans, food-cans, aerosols, aluminium-foil, metal-lids, pop-tabs
Paper (ID 2)	Cardboard-boxes, drink-cartons, egg-cartons, paper-bags, tissues, newspaper
Plastic (ID 3)	PET-bottles, plastic-bags, bubble-wrap, crisp-packets, plastic-cups, food-wrap
Trash (ID 4)	Batteries, cigarettes, food-waste, face-masks, blister-packs, unlabeled-litter

Wesentliche Tabellen der Experimente

Tabelle 5: Vergleich der Klassifikationsmodelle

Modell	Val. Accuracy	Val. Loss	Latenz (CPU)	Dateigröße
EXP-001 Baseline CNN	61.00 %	1,0633	5.81 ms	15.22 MB
EXP-002 Frozen MobileNetV2	84.28 %	0,5659	38.00 ms	8.49 MB
EXP-003 Fine-Tuned MobileNetV2	87.42 %	0,3148	40.00 ms	23.48 MB
EXP-004 INT8 TFLite	87.35 %	–	10.32 ms	2.61 MB

Tabelle 6: Detektionsresultate: Architektur- und Datensatzvergleich (EXP-013 bis EXP-016)

Experiment	Datensatz	mAP50	mAP50-95	Latenz (GPU)	Latenz (CPU)
EXP-013 YOLO11n	TACO+TrashNet (mira_v2)	55.10 %	49.80 %	3.60 ms	45.20 ms
EXP-014 YOLO11n	TACO+TrashNet+Roboflow	60.70 %	50.60 %	1.80 ms	46.00 ms
EXP-015 YOLO11n	TACO+TrashNet+WaRP	56.00 %	45.10 %	1.50 ms	45.50 ms
EXP-016 YOLO11n	WaRP-only	58.80 %	43.20 %	1.20 ms	44.80 ms

Hypothesenüberprüfung

Tabelle 7: Klassenspezifische Metriken von EXP-014 (YOLO11n, TACO+TrashNet+Roboflow)

Klasse	Precision	Recall	mAP50	mAP50-95	Instanzen
Glass	0.580	0.521	0.502	0.400	336
Metal	0.670	0.699	0.713	0.613	439
Paper	0.820	0.772	0.829	0.745	474
Plastic	0.715	0.699	0.721	0.601	1316
Trash	0.523	0.230	0.269	0.173	561

Tabelle 8: Modellvergleich im realen Praxistest (Field Benchmark)

Modell	TP	FP	FN	Precision	Recall	F1-Score
mira_exp014.pt	669	51	170	92.9 %	79.7 %	85.8 %
mira_exp014_int8.tflite	492	20	347	96.1 %	58.6 %	72.8 %
mira_exp015.pt	572	68	267	89.4 %	68.2 %	77.3 %
mira_exp015_int8.tflite	579	67	260	89.6 %	69.0 %	78.0 %
mira_exp013.pt	589	82	250	87.8 %	70.2 %	78.0 %
mira_exp013_int8.tflite	475	30	364	94.1 %	56.6 %	70.7 %
mira_exp011.pt	624	94	215	86.9 %	74.4 %	80.2 %
mira_exp009_int8.tflite	624	107	215	85.4 %	74.4 %	79.5 %
mira_exp006.pt	696	139	259	83.4 %	72.9 %	77.8 %
mira_exp006_int8.tflite	375	197	473	65.6 %	44.2 %	52.8 %
mira_exp011_int8.tflite	0	0	839	0.0 %	0.0 %	0.0 %

Tabelle 9: Klassenspezifische F1-Scores von EXP-014 im Praxistest

Klasse	Precision	Recall	F1-Score
Glass	93.0 %	86.9 %	89.9 %
Metal	89.2 %	81.1 %	84.9 %
Paper	94.6 %	87.2 %	90.7 %
Plastic	96.8 %	82.3 %	88.9 %
Trash	75.5 %	42.0 %	54.0 %

Tabelle 10: Systematische Überprüfung und Verifikation der Forschungshypothesen

Hypothese	Zielvorgabe	Gemessener Wert	Status
H1 (Transfer Learning)	≥ 15 Prozentpunkte Genauigkeitsgewinn ggü. Scratch-CNN	+26,42 pp (EXP-003: 87,42% vs. EXP-001: 61,00%)	Bestätigt
H2 (Modellkompression)	Modellschnitt $\geq 4x$ kleiner, Latenzreduktion $\geq 40\%$, Genauigkeitsverlust < 2 pp	9,0x kleiner (2,61 MB vs. 23,48 MB), 74,2% Latenzsenkung (10,32 ms vs. 40 ms), - 0,07 pp Verlust	Bestätigt
H3 (Multi-Objekt-Erkennung)	Lokalisierung mehrerer Objekte, Inferenzrate ≥ 15 FPS auf CPU	21,8 FPS (ca. 46.0 ms CPU-Latenz mit YOLO11n, EXP-014)	Bestätigt
H4 (EMA-Signalglättung)	Verhinderung von mechanischem Jitter bei $\alpha = 0,15$ ohne kritischen Verzug	Signalfluktuationen gedämpft, stochastisches Rauschen eliminiert, Verzug nur ca. 250 ms	Bestätigt

Quellen- und Literaturverzeichnis

- Ben-David, Shai u. a. (2010). „A Theory of Learning from Different Domains“. In: *Machine Learning* 79.1-2, S. 151–175.
- Bengio, Yoshua, Aaron Courville und Pascal Vincent (2013). „Representation Learning: A Review and New Perspectives“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, S. 1798–1828.
- Deng, Lei u. a. (2020). „Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey“. In: *Proceedings of the IEEE* 108.4, S. 485–532.
- Jocher, Glenn, Ayush Chaurasia und Jing Qiu (2023). *Ultralytics YOLO*. <https://github.com/ultralytics/ultralytics>. Softwareversion 8.0 / 11.0, Zugriff: 07. Juli 2026.
- Kirillov, Alexander u. a. (2023). „Segment anything“. In: *arXiv preprint arXiv:2304.02643*.
- Krishnamoorthi, Raghuraman (2018). „Quantizing deep convolutional networks for efficient inference: A whitepaper“. In: *arXiv preprint arXiv:1806.08342*.
- LeCun, Yann, Yoshua Bengio und Geoffrey Hinton (2015). „Deep learning“. In: *Nature* 521.7553, S. 436–444.
- Proenca, Pedro F. und Pedro Simoes (2020). „TACO: Trash Annotations in Context for Litter Detection“. In: *arXiv preprint arXiv:2003.06975*.
- Redmon, Joseph u. a. (2016). „You Only Look Once: Unified, Real-Time Object Detection“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, S. 779–788.
- Ren, Shaoqing u. a. (2015). „Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks“. In: *Advances in Neural Information Processing Systems*, S. 91–99.
- Roboflow (2023). *Trash Detection Dataset*. <https://universe.roboflow.com/>. Zugriff: 07. Juli 2026. (Besucht am 07. 07. 2026).
- Sandler, Mark u. a. (2018). „MobileNetV2: Inverted Residuals and Linear Bottlenecks“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, S. 4510–4520.
- TensorFlow Authors (2024). *Transfer Learning and Fine-Tuning*. https://www.tensorflow.org/tutorials/images/transfer_learning. Zugriff: 07. Juli 2026. (Besucht am 07. 07. 2026).
- Umweltbundesamt (2023). *Aufkommen und Verwertung von Verpackungsabfällen in Deutschland*. <https://www.umweltbundesamt.de/daten/ressourcen-abfall/verwertung-entsorgung-ausgewaehlter-abfallarten/verpackungsabfaelle>. Zugriff: 07. Juli 2026.
- Wang, Xiang, Yujie Zhang, Ran Wang u. a. (2020). „A Survey on TinyML: Hardware, Software, and Applications“. In: *IEEE Communications Surveys & Tutorials* 24.4, S. 2127–2160.
- WaRP Project (2024). *Waste Recycling Plant Dataset (WaRP)*. <https://github.com/WaRP-DS/WaRP>. Zugriff: 07. Juli 2026. (Besucht am 07. 07. 2026).
- Yang, Mindy und Gary Thung (2016). *Classification of trash for recyclability status*. Techn. Ber. CS229 Project Report, Stanford University.
- Yosinski, Jason u. a. (2014). „How transferable are features in deep neural networks?“ In: *Advances in Neural Information Processing Systems*, S. 3320–3328.
- Zha, Daochen u. a. (2023). „Data-centric Artificial Intelligence: A Survey“. In: *arXiv preprint arXiv:2303.10158*.